

Paper Critique: SWE-agent and AutoCodeRover

Papers:

- Yang et al., *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*, NeurIPS '24
- Zhang et al., *AutoCodeRover: Autonomous Program Improvement*, ISSTA '24

Both papers tackle SWE-bench: given a real GitHub issue and its repository, produce a patch that passes the hidden tests. SWE-agent gives the LLM a carefully designed interactive environment, while AutoCodeRover gives it program-analysis tools inside a fixed two-phase workflow. I also ran each tool’s current release with a local 1.5B model (Qwen2.5-Coder), which says nothing about the papers’ success rates but does show which parts of each design are robust.

1 What I liked

Both papers target the task developers actually do (read a vague issue, find the relevant code in a large repo, and fix it without breaking anything) rather than HumanEval-style single functions. SWE-agent’s Agent-Computer Interface (compact actions, concise feedback, context management, and a linter guardrail that rejects invalid edits) is a new idea, and its ablations show that changing the interface alone moves the resolve rate by several points with the model held fixed; in my run the guardrail rejected a syntax-error edit and restored the file byte for byte. AutoCodeRover treats the repository as a program, using AST-aware search APIs and spectrum-based fault localization to reach about 19% on SWE-bench Lite at about \$0.43 per issue, and in my run `search_method_in_class("split", "Batcher")` found the right method where `grep` would have misled. For well-specified bugs in well-tested Python repos this already partly works, and maintainers with issue backlogs and companies with large codebases want it now, though “assign any ticket to an agent” is still a few years off. The most controversial point is that the papers can’t both be right about the main lever (interface vs. program analysis), and their cross-paper comparisons use different models, budgets and settings.

2 What I disliked

Both are evaluated almost entirely on SWE-bench, which risks training-data contamination and has weak tests, and results are single runs at temperature 0 where a 1–2 point gap on about 300 tasks is only a handful of issues. The headline comparisons are not controlled, because the two methods are never run on the same model, prompts and budget. SWE-agent’s ACI is tuned to GPT-4 and fragile: my local model emitted multi-command scripts, invented an edit range for an 8-line file, and the parser dropped the entire heredoc edit body. AutoCodeRover’s fixed search-then-patch workflow assumes the issue names the relevant code, and its tools had flaws in my run (4 matches for 2 definitions, and a crash on Windows from Unix-only `signal.SIGALRM`). Both artifacts have also

drifted from the papers: SWE-agent crashed on a SWE-ReX version mismatch, and AutoCodeRover hit `NotImplementedError` yet still exited with code 0.

3 Future directions for this research

The authors propose extending ACIs to other domains and learning them automatically (SWE-agent), and stronger fault localization, test generation and more languages (AutoCodeRover). My main idea is to combine the two: an ACI whose navigation actions are AST queries such as `open_method` `Batcher.split` or `callers_of X`, tested in a controlled ablation on the same model. The interface could also enforce its own action format through constrained decoding or by rejecting multi-command responses, which would make these agents usable with small local models for privacy-sensitive companies. The same approach could target vulnerability repair, API migrations or flaky tests, and Agentless-style results suggest studying how much autonomy each class of issue really needs. Evaluations should add post-cutoff repositories, non-Python languages, repeated runs, cost-normalized comparisons, and a user study of whether maintainers accept the patches.

4 Open questions

How much of SWE-agent’s gain comes from the ACI and how much from GPT-4 already knowing shell and editor conventions, and would the ablations hold on an open-weights model? When AutoCodeRover’s localization fails, is it because the search APIs returned the wrong thing or because the LLM asked the wrong questions? If a patch passes the tests but differs in meaning from the developer’s fix, is it wrong, and how should overfitting to weak tests be handled? How do these systems cope with bugs that have no clear location, such as performance, concurrency or build issues? Finally, are “issue → patch” tasks representative of real maintenance, where many issues are feature requests or need design discussion first?

5 Most important for class discussion

The central question is where the improvement really comes from (a better interface, better program analysis, or simply a better model) and what experiment would separate the three. We should also discuss how far to trust SWE-bench numbers and what minimum evaluation standard agent papers should meet. Reproducibility deserves attention too: in my runs neither tool produced a patch, partly because of the weak model and partly because of fragile frameworks. I’d like the professor’s view on whether free-roaming agents or fixed, analysis-driven pipelines are the better foundation for industry tools. Lastly, if agents resolve a meaningful share of issues, does the maintainer’s job shift toward specifying and verifying fixes, and what should SE education emphasize in response?

6 Summary

Both papers show that LLMs get much better at repository-level bug fixing when given the right tools instead of just a bigger prompt. SWE-agent shows that the interface between model and

computer is a first-class design problem, and AutoCodeRover shows that program-analysis research still makes LLM repair cheaper and more targeted. These designs amplify a capable model but don't replace one: with a small local model, neither tool fixed an easy bug, because of failures in both the model and the tools. Both are also evaluated narrowly, and their artifacts have drifted since publication. The lasting principle is to design the agent's environment as carefully as a tool for a human developer, grounded in real program structure.